Звіт з лабораторної роботи №4

Навчальна дисципліна: Web-програмування **Тема:** Робота з функціями у JavaScript **Виконав:** [ПІБ], група [___], варіант [1 або 2 — залиште один] **Перевірив:** [ПІБ викладача]

5.1 Мета роботи

Ознайомитися з принципами роботи та типами функцій у JavaScript, включаючи іменовані функції, функціональні вирази, стрілкові функції, замикання, рекурсію та колбеки.

5.2 Теоретичні відомості

Функції — основний будівельний блок JavaScript: вони організовують код, підвищують його читабельність і дають змогу використовувати його повторно.

Способи оголошення функцій.

- *Іменована функція (Function Declaration)* — `function greet(name) { ... }`. Піднімається (hoisting), тому її можна викликати до оголошення.
- *Функціональний вираз (Function Expression)* — функція зберігається у змінній: `const sum = function (a, b) { return a + b; };`. Не піднімається.
- *Стрілкова функція (Arrow Function)* — `const multiply = (a, b) => a * b;`. Має компактний синтаксис, не створює власного `this` і не має об'єкта `arguments`.

Параметри та аргументи. Параметри можуть мати значення за замовчуванням (`name = "Гість"`). У звичайних функціях доступний об'єкт `arguments`. Оператор залишкових параметрів `...rest` збирає невизначену кількість аргументів у масив і має стояти останнім у списку параметрів. Оператор `...spread` розгортає масив в окремі аргументи під час виклику.

Колбеки (callback) — функції, які передаються як аргументи в інші функції для відкладеного виконання; застосовуються в асинхронному коді (`setTimeout`) та в методах масивів (`map`, `forEach`).

Вкладені функції та область видимості. Внутрішня функція має доступ до змінних зовнішньої.

Замикання (closure) — функція, яка зберігає доступ до змінних батьківської області видимості навіть після завершення її виконання.

Рекурсія — функція, що викликає саму себе; обов'язково має базовий випадок. Надто глибока рекурсія призводить до переповнення стека.

Функції вищого порядку приймають інші функції як аргументи або повертають функції як результат.

IIFE — самовиконувані функції `(function () { ... })();`, що створюють локальну область видимості, ізолюють змінні й виконують код один раз.

5.3 Завдання

1. Створити гілку `lab4`, додати до проєкту `lab4/index.html` та `lab4/main.js`.
2. Відкрити сторінку та Developer Tools, виконати в `main.js` завдання свого варіанта (непарний номер у журналі — Варіант 1, парний — Варіант 2).
3. Додати посилання на `lab4/index.html` у меню `labs/index.html`.
4. Злити гілку `lab4` в основну (merge) та виконати push.
5. Переконатися, що проєкт оновився на Netlify, скопіювати посилання у звіт.
6. Оформити звіт, відповісти на питання вихідного контролю, зробити висновки.

Варіант 1: `calculateSum(n)` (сума від 1 до n); функціональний вираз `multiply(a, b)`; стрілкова `power(a, b)`; рекурсивна `harmonicSeries(n)`; замикання `createMultiplier(multiplier)`; `applyFunction(value, func)` (приклад: подвоєння та квадрат); `processSet(set, callback)` *.

Варіант 2: `findMax(a, b)`; функціональний вираз `subtract(a, b)`; стрілкова `sqrt(n)`; рекурсивна `geometricProgression(n, a, r)`; замикання `createDivider(divisor)`; `applyOperation(a, b, func)` (приклад: додавання та множення); `processSet(set, callback)` *.

5.4 Звіт по роботі

Підготовка проєкту (пп. 3.1–3.3)

```
cd labs
git checkout -b lab4
mkdir lab4
```

У папці `lab4` створено файли `index.html` та `main.js`.

`lab4/index.html`:

```
<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lab 4 — Робота з функціями у JavaScript</title>
</head>
<body>
  <h1>Лабораторна робота №4</h1>
  <p>Відкрийте Developer Tools (F12) → Console, щоб побачити результати.</p>
  <script src="main.js"></script>
</body>
</html>
```

Виконання завдань — Варіант 1 (п. 3.4)

```
// Завдання 1: сума чисел від 1 до n
function calculateSum(n) {
  let sum = 0;
  for (let i = 1; i <= n; i++) {
    sum += i;
  }
  return sum;
}

// Завдання 2: функціональний вираз
const multiply = function (a, b) {
  return a * b;
};

// Завдання 3: стрілкова функція
const power = (a, b) => a ** b;

// Завдання 4: рекурсія, H(n) = 1 + 1/2 + ... + 1/n
function harmonicSeries(n) {
  if (n <= 1) {
    return 1;                              // базовий випадок
  }
  return 1 / n + harmonicSeries(n - 1);    // рекурсивний виклик
}

// Завдання 5: замикання
function createMultiplier(multiplier) {
  return function (number) {
    return number * multiplier;
```

```
  };
}
const double = createMultiplier(2);
const triple = createMultiplier(3);

// Завдання 6: функція приймає функцію
function applyFunction(value, func) {
  return func(value);
}
applyFunction(7, (x) => x * 2);   // подвоєння
applyFunction(7, (x) => x * x);   // квадрат

// Завдання 7*: колбек для всіх елементів Set
function processSet(set, callback) {
  const result = new Set();
  set.forEach((item) => result.add(callback(item)));
  return result;
}
processSet(new Set([1, 2, 3, 4]), (x) => x * 10);
```

Результат у консолі:

Завдання	Виклик	Результат
1	`calculateSum(10)`	55
2	`multiply(6, 7)`	42
3	`power(2, 10)`	1024
4	`harmonicSeries(5)`	2.283333333333333
5	`double(5)`; `triple(5)`	10; 15
6	подвоєння 7; квадрат 7	14; 49
7*	`processSet(new Set([1,2,3,4]), x => x * 10)`	Set {10, 20, 30, 40}

[Скриншот 1: вкладка Console, результати Варіанта 1]

Виконання завдань — Варіант 2 (п. 3.4)

```
// Завдання 1: максимум із двох чисел
function findMax(a, b) {
  return a > b ? a : b;
}
```

```
// Завдання 2: функціональний вираз
const subtract = function (a, b) {
  return a - b;
};

// Завдання 3: стрілкова функція
const sqrt = (n) => Math.sqrt(n);

// Завдання 4: рекурсія, S(n) = a * (1 - r^n) / (1 - r)
function geometricProgression(n, a, r) {
  if (n <= 0) {
    return 0;                                                    // базовий
випадок
  }
  return a * r ** (n - 1) + geometricProgression(n - 1, a, r);  //
рекурсивний виклик
}

// Завдання 5: замикання
function createDivider(divisor) {
  return function (number) {
    return number / divisor;
  };
}
const half = createDivider(2);
const third = createDivider(3);

// Завдання 6: функція приймає два числа та функцію
function applyOperation(a, b, func) {
  return func(a, b);
}
applyOperation(5, 3, (x, y) => x + y);   // додавання
applyOperation(5, 3, (x, y) => x * y);   // множення

// Завдання 7*: колбек для всіх елементів Set
function processSet(set, callback) {
  const result = new Set();
  set.forEach((item) => result.add(callback(item)));
  return result;
}
processSet(new Set([1, 2, 3, 4]), (x) => x * 10);
```

Результат у консолі:

Завдання	Виклик	Результат
1	`findMax(8, 15)`	15
2	`subtract(10, 4)`	6
3	`sqrt(144)`	12
4	`geometricProgression(5, 2, 3)`	242 (збігається з $2\cdot(1-3^5)/(1-3)$)
5	`half(10)`; `third(9)`	5; 3
6	додавання 5 і 3; множення 5 і 3	8; 15
7*	`processSet(new Set([1,2,3,4]), x => x * 10)`	Set {10, 20, 30, 40}

[Скриншот 2: вкладка Console, результати Варіанта 2]

Меню, merge, push і Netlify (пп. 3.5–3.7)

У `labs/index.html` додано посилання:

```
<li><a href="lab4/index.html">Лабораторна робота 4</a></li>
```

```
git add .
git commit -m "lab4: functions in JavaScript"
git checkout main
git merge lab4
git push origin main
```

Після push Netlify автоматично оновив сайт.

Посилання на проєкт на Netlify: [вставте URL, напр. https://ваш-сайт.netlify.app/lab4/index.html]

[Скриншот 3: сторінка lab4 на Netlify]

Відповіді на питання вихідного контролю

4.1. Які існують способи оголошення функцій у JavaScript?

Три основні: іменована функція (`function name() {}`), функціональний вираз (`const f = function () {}`) та стрілкова функція (`const f = () => {}`). Також існують анонімні функції, що передаються як аргументи, і IIFE.

4.2. Що таке функціональний вираз? Чим він відрізняється від іменованої функції?

Функціональний вираз — це функція, оголошена як вираз і присвоєна змінній, наприклад `const sum = function (a, b) { return a + b; };`. Іменована функція піднімається (hoisting), тому її можна викликати до оголошення в коді; функціональний вираз не піднімається і доступний лише після рядка, де він визначений.

4.3. Як працює замикання (closure)?

Коли функція створюється всередині іншої, вона запам'ятовує змінні її області видимості. Тому після завершення зовнішньої функції внутрішня все ще має до них доступ. У роботі: `createMultiplier(multiplier)` повертає функцію, яка «пам'ятає» значення `multiplier`, тому `double(5)` дає 10, а `triple(5)` — 15 (кожне замикання має власну копію змінної).

4.4. Що таке колбек-функція? Наведіть приклади використання.

Колбек — функція, передана як аргумент в іншу функцію й викликана нею пізніше. Приклади: `applyFunction(7, x => x * 2)`; `processSet(set, callback)`; `numbers.map(num => num * 2)`; асинхронно — `setTimeout(() => callback("Дані отримано"), 2000)`.

4.5. Як реалізувати рекурсію у JavaScript?

Функція викликає саму себе з простішими аргументами й обов'язково має базовий випадок, що зупиняє рекурсію. Приклад з роботи: `harmonicSeries(n)` (Варіант 1) повертає 1 при `n <= 1`, інакше `1 / n + harmonicSeries(n - 1)`; `geometricProgression` (Варіант 2) повертає 0 при `n <= 0`. Без базового випадку або при надто великій глибині виникає переповнення стека (`RangeError`).

4.6. Як працює оператор `...` (rest) при оголошенні функцій?

У списку параметрів `...rest` збирає всі решту аргументів у справжній масив і має бути останнім параметром: `function f(first, second, ...rest)`. Виклик `f(1, 2, 3, 4, 5)` дає `first = 1`, `second = 2`, `rest = [3, 4, 5]`. На відміну від `arguments`, працює і в стрілкових функціях.

4.7. Як працює оператор `...` (spread) у виклику функції при передаванні масиву?

У виклику `...` розгортає елементи масиву в окремі аргументи: для `const numbers = [1, 2, 3, 4]` виклик `test(...numbers)` рівнозначний `test(1, 2, 3, 4)`. Так само він використовується для копіювання й об'єднання масивів.

Висновки

У ході лабораторної роботи опрацьовано основні види функцій JavaScript: іменовані функції, функціональні вирази та стрілкові функції. Реалізовано функції з рекурсією (базовий випадок і рекурсивний виклик), замикання (`createMultiplier` / `createDivider`), функції вищого порядку та колбеки, зокрема обробку елементів колекції `Set`. Результати перевірено у консолі Developer Tools. Замикання дозволяють зберігати стан між викликами, колбеки й функції вищого порядку роблять код гнучким та придатним до повторного використання, а рекурсія зручна для задач, що розкладаються на однотипні підзадачі, проте потребує обережності через можливе переповнення стека. Роботу додано до основної гілки репозиторію та опубліковано на Netlify.

Перелік посилань

1. Mozilla Developer Network (MDN). «Functions — reusable blocks of code». Документація.
2. Кайл Сімпсон. «Розумний JavaScript», 2021.
3. Фленаган Д. «JavaScript. Повне керівництво».
4. ECMAScript Language Specification. «Functions and Classes». Специфікація.